

Decomposing Algebraic Numbers into Low-Degree Sums and Products

Exact algorithms, global degree bounds,
and a Wolfram Language reference implementation

A solution study for Mathematica StackExchange question 105933

6 September 2026

Abstract

Given an exact algebraic number, we seek a flat sum or product of algebraic numbers whose largest absolute degree is as small as possible. The two ninth-degree examples in the question have globally optimal decompositions into the specified pair of cubic roots. We prove their identities by explicit polynomial congruences and prove optimality against arbitrarily many factors or terms, including ones outside the input field. We then develop a template-free algorithm: factor over the input number field, recover every subfield by rational linear algebra, and solve additive membership or multiplicative intersection problems. A normal-closure averaging argument makes additive minimization globally complete. Products admit an exact binary algorithm, an exact tensor test for linearly disjoint families, and a bounded exhaustive search. We distinguish these guarantees carefully from a general unrestricted multiplicative minimax algorithm. The accompanying archive contains the complete Wolfram source, local regression tests, and independently executed exact SymPy verification.

What is established. Both examples are solved with a proved optimum of 3. The additive problem has a complete finite algorithm, subject in practice to explicit resource limits. The product implementation is complete in the specified binary, independent-family, or bounded-search classes; it is *not* presented as a terminating solver for every unrestricted global multiplicative minimization problem.

Execution status. The mathematical certificates and an independent SymPy implementation were executed successfully. A Wolfram connector failed with HTTP 404 before evaluation, so the supplied Wolfram package and test suite have not been runtime-tested in Mathematica.

Source files: `RootDecomposition.wl`, `tests.wlt`, `examples.wl`,
`verify_exact.py`, and `verification_results.json`.

Contents

1	The two examples: answers and an optimality certificate	3
1.1	Immediate Mathematica use	3
1.2	Why maximum degree two is impossible	4
1.3	Exact recovery polynomials	4
1.4	Degree and branch identification without numerical guessing	4
2	What is being minimized?	5
3	Universal lower bounds	5
4	Resultants: forward construction and inverse templates	6
4.1	The two basic identities	6
4.2	What coefficient matching does and does not prove	7
5	Recovering every subfield by exact linear algebra	7
5.1	Principal fields and completeness	8
5.2	The actual rational matrix	8
5.3	Wolfram coordinate discipline	9
6	A complete additive algorithm	9
6.1	All numbers in one finite ambient field	9
6.2	Presentation and number of terms	10
6.3	Descent to a normal closure	10
6.4	Constructing and optimizing in that field	11
7	Products of two factors: a linear, not nonlinear, test	11
8	Several factors and the exact tensor test	12
8.1	Independent field families	12
8.2	A three-factor example that defeats binary minimization	13
8.3	Exactly what the package searches	13
9	Why restricting to the input field can miss the optimum	13
9.1	Deriving the sextics	14
9.2	Galois group and field lattice	14
9.3	Practical consequences	15
10	A complete finite-box search for either operation	15
11	Implementation contracts, usage, and resource limits	16
11.1	Public functions	16
11.2	Reusing an ambient field	17
11.3	Resource guards and asymptotic bottlenecks	17
11.4	Output quality versus degree optimality	18
12	Executed verification and reproducibility	18
12.1	What was actually run	18
12.2	What was not run	19
12.3	Reproduction commands	19
13	Conclusions	19
A	Full Wolfram Language reference implementation	20
	References	26

1 The two examples: answers and an optimality certificate

Write

$$P(x) = x^3 + x + 1, \quad Q(x) = x^3 - x + 1, \quad (1)$$

$$f_{\times}(x) = x^9 + 2x^7 - 3x^6 + x^5 - x^4 + 3x^3 - x - 1, \quad (2)$$

$$f_{+}(x) = x^9 + 6x^6 + 3x^5 - 15x^3 + 24x^2 - 4x + 8. \quad (3)$$

Let a and b be the real roots of P and Q , respectively. They are the two cubic `Root` objects requested in the question. The answers are

$$\boxed{\alpha_{\times} = ab, \quad \alpha_{+} = a + b, \quad D_{\times}(\alpha_{\times}) = D_{+}(\alpha_{+}) = 3.} \quad (4)$$

Here $\alpha_{\times}$ and α_{+} denote the real roots of (2) and (3). Their absolute degrees are both nine. The notation D_{+} and $D_{\times}$ is defined precisely in Section 2.

1.1 Immediate Mathematica use

The following is an exact identity check, not a numerical recognition test:

```
a = Root[1 + # + #^3 &, 1];
b = Root[1 - # + #^3 &, 1];
ap = Root[-1 - # + 3 #^3 - #^4 + #^5 - 3 #^6 +
  2 #^7 + #^9 &, 1];
as = Root[8 - 4 # + 24 #^2 - 15 #^3 + 3 #^5 +
  6 #^6 + #^9 &, 1];

{RootReduce[ap - a b], RootReduce[as - a - b]}
(* Both entries are mathematically zero. *)
```

`RootReduce` is the appropriate exact certification primitive: for algebraic expressions built from exact roots it produces a canonical algebraic result, with simple cases simplifying to rationals or radicals [5]. Equal numerical approximations or equal minimal polynomials alone do not establish the desired branch identity.

To *discover* the decompositions without supplying a or b , load the package accompanying this article:

```
Get["/your/path/RootDecomposition.wl"];

rp = ProductDecomposition[ap];
rs = GlobalAdditiveDecomposition[as];

DecompositionExpression[rp]
DecompositionExpression[rs]
{rp["MaximumDegree"], rs["MaximumDegree"]}
{rp["GlobalOptimalityCertified"], rs["GlobalOptimalityCertified"]}
```

The proved mathematical targets for the last two lines are $\{3, 3\}$ and $\{\text{True}, \text{True}\}$. These are expected results of the reference implementation, not a claimed transcript from a Wolfram kernel. Factors and terms are stored separately in the returned association; `DecompositionExpression` uses an inactive operation to preserve that structure. The order of the two cubics is immaterial.

1.2 Why maximum degree two is impossible

Every algebraic number of degree at most two belongs to a quadratic extension of $\mathbb{Q}$, or to $\mathbb{Q}$ itself. A field generated by any finite collection of such numbers is multiquadratic: its degree over $\mathbb{Q}$ is a power of two. Every element of that field has degree dividing that power of two. Consequently no number of degree nine can be a finite sum or product of numbers of degree at most two. The displayed cubic decompositions attain three, so three is the *global* optimum. This proof places no restriction on the number of terms or on their belonging to the input field.

Section 3 generalizes this obstruction. For the present examples it is already sufficient, once degree nine is certified.

1.3 Exact recovery polynomials

There is a particularly transparent certificate for the product. Define

$$A_{\times}(x) = -\frac{x^7 + 2x^5 - 2x^4 + 1}{2}, \quad (5)$$

$$B_{\times}(x) = -\frac{x^7 + 2x^5 - 2x^4 + 2x^3 + 2x - 1}{2}. \quad (6)$$

Direct polynomial division over $\mathbb{Q}$ gives

$$\begin{aligned} A_{\times}^3 + A_{\times} + 1 &\equiv 0, \\ B_{\times}^3 - B_{\times} + 1 &\equiv 0, \quad (\text{mod } f_{\times}(x)). \\ A_{\times}B_{\times} - x &\equiv 0 \end{aligned} \quad (7)$$

Thus the cubics are recovered from the input alone as $a = A_{\times}(\alpha_{\times})$, $b = B_{\times}(\alpha_{\times})$.

For the sum define

$$A_{+}(x) = -\frac{1}{83732}(2025x^8 + 750x^7 - 4374x^6 + 10530x^5 + 9975x^4 - 28868x^3 - 51921x^2 - 12496x + 39992), \quad (8)$$

and put $B_{+}(x) = x - A_{+}(x)$. Then

$$A_{+}^3 + A_{+} + 1 \equiv 0, \quad B_{+}^3 - B_{+} + 1 \equiv 0, \quad A_{+} + B_{+} - x = 0 \quad (\text{mod } f_{+}(x)). \quad (9)$$

These are rational polynomial certificates, entirely independent of root approximation. All six congruences were checked by the included Python verification. They can also be checked locally with `PolynomialRemainder`; the complete expressions appear in `tests.wlt`.

1.4 Degree and branch identification without numerical guessing

The cubics are irreducible by the rational-root test. Their discriminants are -31 and -23 , so each has one real root and its splitting field has Galois group S_3 . The two splitting fields have trivial intersection. Indeed, a nontrivial proper common Galois quotient of two S_3 -extensions would give their unique quadratic subfields, respectively $\mathbb{Q}(\sqrt{-31})$ and $\mathbb{Q}(\sqrt{-23})$, which are different; equality of the full splitting fields would also force equality of those quadratic subfields. Therefore the splitting fields are linearly disjoint, and the compositum of the two chosen cubic fields has degree nine. The Galois correspondence and compositum facts used here are standard; see [3, Chapters 3–4].

The resultant identities proved in Section 4 show that ab and $a + b$ satisfy $f_{\times}$ and f_{+} . Applying (7) or (9) at those real numbers recovers the unique real roots of P and Q . Hence each ninth-degree expression generates the degree-nine compositum. Both displayed nonics are therefore minimal polynomials and irreducible.

Moreover any real root t of either nonic is sent by its recovery polynomials to real roots of P and Q , which must be a and b . The last congruence then forces $t = ab$ or $t = a + b$, respectively. Thus each nonic has exactly one real root. Mathematica orders the real roots first, increasingly [4]; the index 1 in both inputs is therefore correct. The independent verification also checks irreducibility and exact real-root counts directly.

2 What is being minimized?

Definition 2.1. For an algebraic number γ , its *absolute degree* is

$$D(\gamma) = [\mathbb{Q}(\gamma) : \mathbb{Q}].$$

For an algebraic α , define

$$D_{+}(\alpha) = \min_{\substack{r \geq 1 \\ \alpha = \beta_1 + \dots + \beta_r}} \max_i D(\beta_i), \quad D_{\times}(\alpha) = \min_{\substack{r \geq 1 \\ \alpha = \beta_1 \dots \beta_r}} \max_i D(\beta_i).$$

The terms and factors range over $\overline{\mathbb{Q}}$. For nonzero α the factors are necessarily nonzero. We use $\{0\}$ as the trivial product representation of zero. Rationals, including zero, have degree one.

The singleton representation $\{\alpha\}$ always exists, so both minima exist and lie between one and $D(\alpha)$. Requiring at least two entries would not change them: one can append zero to a sum or one to a product. This is a minimax problem, not a minimization of expression length.

The coefficients of each defining polynomial must be rational. Otherwise one could put $x - \alpha$ into a degree-one **Root**, making the question vacuous. Likewise a nested radical with low *relative* degrees is not necessarily a small-absolute-degree decomposition. For example, a square root of a quadratic irrational can have degree four over $\mathbb{Q}$.

We distinguish three domains of search. An *internal* decomposition requires every term or factor to belong to $K = \mathbb{Q}(\alpha)$. An *ambient-field* decomposition permits a specified finite extension K containing α . A *global* decomposition permits all algebraic numbers. These are not interchangeable: Section 9 gives explicit examples with internal optimum six and global optimum four.

For an exact **Root**, use **MinimalPolynomial** before measuring degree. The polynomial originally written inside a root expression need not be irreducible. Mathematica's primitive integral minimal-polynomial convention also supplies a consistent coefficient-height measure for the bounded search [6].

3 Universal lower bounds

Theorem 3.1 (Largest-prime-factor bound). *Let $n = D(\alpha)$, and put $\text{lpf}(1) = 1$. Then*

$$D_{+}(\alpha) \geq \text{lpf}(n), \quad D_{\times}(\alpha) \geq \text{lpf}(n).$$

More generally, if α belongs to a field generated by finitely many algebraic numbers of degree at most d , every prime divisor of $D(\alpha)$ is at most d .

Proof. Let the generators have degrees $m_1, \dots, m_r \leq d$, and let N_i be their normal closures. The group $\text{Gal}(N_i/\mathbb{Q})$ embeds into S_{m_i} , so its order divides $m_i!$. The Galois group of the compositum $N = N_1 \cdots N_r$ embeds into the product of those groups. Therefore every prime divisor of $[N : \mathbb{Q}]$ is at most d . Since $\mathbb{Q}(\alpha) \subseteq N$, the tower formula gives $D(\alpha) \mid [N : \mathbb{Q}]$. Any sum or product of the generators belongs to that compositum, proving the assertions. $\square$

For prime degree n , neither operation can lower the maximum degree at all. For degree nine the bound is three; for degree eight it is two. The bound may be weaker than the true optimum, but it is inexpensive and remains valid with arbitrarily many factors outside the input field.

A tempting stronger statement, $D(\alpha) \mid \prod_i D(\beta_i)$, is *false* in general. Two distinct conjugate roots of an irreducible S_3 -cubic can generate its degree-six splitting field although the product of their individual degrees is nine. The safe general statements are the normal-closure divisibility used above and $[\mathbb{Q}(\beta_1, \dots, \beta_r) : \mathbb{Q}] \leq \prod_i D(\beta_i)$.

Proposition 3.2 (Galois-exponent obstruction). *Let G be the Galois group of the normal closure of $\mathbb{Q}(\alpha)$. If α is a sum or product of numbers of degree at most d , then*

$$\exp(G) \mid \text{lcm}(1, 2, \dots, d).$$

Proof. The exponent of S_m is $\text{lcm}(1, \dots, m)$, because permutation orders are least common multiples of cycle lengths. In the notation of the previous proof, the exponent of $\text{Gal}(N/\mathbb{Q})$, a subgroup of a product of such symmetric groups, divides $\text{lcm}(1, \dots, d)$. The normal closure of $\mathbb{Q}(\alpha)$ is a Galois subfield of N , so G is a quotient of $\text{Gal}(N/\mathbb{Q})$. Quotients cannot increase the exponent. $\square$

In particular an S_4 normal closure rules out degree at most three, because S_4 contains an element of order four, whereas $\text{lcm}(1, 2, 3) = 6$. This sharper obstruction will certify the sextic counterexamples later. The package computes the inexpensive largest-prime bound automatically, not arbitrary Galois groups.

4 Resultants: forward construction and inverse templates

4.1 The two basic identities

Suppose $P, Q \in \mathbb{Q}[x]$ are monic, of degrees r, s , with roots a_i and b_j . Then

$$R_+(x) = \text{Res}_y(P(y), Q(x - y)) = \prod_{i=1}^r \prod_{j=1}^s (x - a_i - b_j), \quad (10)$$

$$R_\times(x) = \text{Res}_y(P(y), y^s Q(x/y)) = \prod_{i=1}^r \prod_{j=1}^s (x - a_i b_j). \quad (11)$$

To prove them, apply the defining product formula for the resultant of a monic polynomial:

$$\text{Res}_y(P(y), H(y)) = \prod_i H(a_i).$$

In (11), the second argument is the polynomial $\sum_{j=0}^s q_j x^j y^{s-j}$; no division by a potentially zero root is required. Wolfram Language supplies `Resultant` directly [11].

A compact implementation is

```

RootSumPolynomial[p_, q_, x_Symbol] := Module[{y = Unique["y"]},
  Expand[Resultant[p /. x -> y, q /. x -> x - y, y]]
];
RootProductPolynomial[p_, q_, x_Symbol] := Module[
  {y = Unique["y"], s = Exponent[q, x], reversed},
  reversed = Sum[Coefficient[q, x, j] x^j y^(s-j), {j, 0, s}];
  Expand[Resultant[p /. x -> y, reversed, y]]
];

```

Substituting $P = x^3 + x + 1$ and $Q = x^3 - x + 1$ gives exactly (3) and (2). These resultant calculations were checked independently with exact rational arithmetic.

The companion-matrix viewpoint is equivalent: the eigenvalues of $C_P \otimes I_s + I_r \otimes C_Q$ are all sums, and those of $C_P \otimes C_Q$ are all products. The original discussion contains inverse coefficient-matching approaches using companion matrices (yarchik) and resultants (Daniel Lichtblau) [1]. Our subfield algorithms below do not require guessing those coefficient templates.

4.2 What coefficient matching does and does not prove

One can choose degrees r, s , introduce rational unknown coefficients in P, Q , and equate the resultant to the input's minimal polynomial. This is often effective when the intended polynomials have very small coefficients. There are, however, several separate requirements.

First, in general one only has

$$f_\alpha(x) \mid R_+(x) \quad \text{or} \quad f_\alpha(x) \mid R_\times(x).$$

Equality follows when the monic polynomials have the same degree. Otherwise there may be several irreducible factors or repeated roots. For instance, using $P = Q = x^2 - 2$ in the sum construction gives $x^2(x^2 - 8)$; using $P = x^2 - 2$, $Q = x^2 - 3$ in the product construction gives $(x^2 - 6)^2$. The chosen sum or product has a minimal polynomial of smaller degree.

Second, a polynomial system solved over $\mathbb{C}$ can return algebraic coefficients for P or Q . Such an answer need not meet the desired *absolute* degree bounds. Rationality of the defining coefficients must be enforced or checked. This is not fixed by merely using a symbolic solver instead of numerical fitting.

Third, the correct root indices still have to be selected. Enumerate roots of the recovered rational polynomials and retain combinations for which `RootReduce[target - expression]` is zero. The resultant encodes all conjugate pairs, not the particular input branch.

Finally, normalization must preserve the rational search space. For sums, replacing (a, b) with $(a - c, b + c)$, $c \in \mathbb{Q}$, allows the trace of one chosen polynomial to be normalized to zero. For products, rescaling (a, b) to $(ca, b/c)$ is safe for rational c , but forcing an arbitrary constant coefficient to become one can require an irrational r -th root. That restriction would exclude valid rational templates.

Thus inverse resultants are useful candidate generators and verification tools, but solving one template is not a proof of global minimax optimality. Factoring an irreducible input polynomial over $\mathbb{Q}$ is not a substitute: it factors a polynomial, not the selected algebraic number as a product.

5 Recovering every subfield by exact linear algebra

The important change of viewpoint is to work in a number field rather than to guess small polynomial coefficients. Let

$$K = \mathbb{Q}(\theta), \quad n = [K : \mathbb{Q}], \quad f(\theta) = 0,$$

where f is the monic minimal polynomial. Elements of K are represented by rational coordinate vectors in the power basis $1, \theta, \dots, \theta^{n-1}$. A subfield is represented by a matrix whose rows are a rational basis of that subfield.

The construction uses *principal subfields*. The fact that all intermediate fields arise as intersections of principal ones, and the use of factorization followed by linear equations, belong to the established subfield literature. Szutkoski and van Hoeij give the formulation and faster partition-based methods [2]. We use a direct rational-matrix version to keep the implementation self-contained; we do not claim their optimized partition algorithm or its performance bounds for this package.

5.1 Principal fields and completeness

Factor f into monic irreducibles over K :

$$f(x) = g_1(x) \cdots g_t(x), \quad g_1(x) = x - \theta.$$

For each factor define

$$L_i = \{h(\theta) : h \in \mathbb{Q}[x], \deg h < n, \quad h(x) - h(\theta) \equiv 0 \pmod{g_i(x)}\}. \quad (12)$$

Proposition 5.1. *Each L_i is a subfield of K . Every intermediate field $\mathbb{Q} \subseteq F \subseteq K$ is an intersection of a subset of the L_i .*

Proof. Let N be a normal closure of K . For each root θ' of g_i , the map sending θ to θ' defines a $\mathbb{Q}$ -embedding $\sigma : K \rightarrow N$. Because g_i is separable, condition (12) says exactly that all these embeddings fix $h(\theta)$. The set of elements fixed by a collection of embeddings is closed under addition, subtraction, multiplication, and inversion of nonzero elements. It contains $\mathbb{Q}$, proving the first assertion.

For an intermediate F , let f_F be the minimal polynomial of θ over F . Viewed in $K[x]$, it is the product of a subset of the factors g_i . An element of F is fixed by every F -embedding of K , so it belongs to the intersection associated to that subset. Conversely, an element in this intersection is fixed by all those embeddings. Every automorphism of N fixing F restricts to one of them; the Galois correspondence then puts the element in F . This proves equality. $\square$

In particular, the field belonging to $x - \theta$ is all of K . We need not construct N to execute this proof's algorithm: the factorization and the linear equations take place entirely in K .

5.2 The actual rational matrix

For one factor g of degree m , compute

$$r_j(x) = \text{rem}_{g(x)}(x^j - \theta^j), \quad 0 \leq j < n.$$

Each r_j has m coefficients in K , padded with zeros as needed. Replace every coefficient by its n -entry rational coordinate vector. Concatenation produces a vector $c_j \in \mathbb{Q}^{mn}$. Then

$$[c_0 \ c_1 \ \cdots \ c_{n-1}]v = 0 \quad (13)$$

is precisely the condition that the element with coordinates v belong to L_g . A rational nullspace therefore gives its basis. There is no search over coefficient heights in this step.

To intersect two row spaces $U, V \subseteq \mathbb{Q}^n$, use

$$U \cap V = (U^\perp + V^\perp)^\perp. \quad (14)$$

Each orthogonal complement is a rational nullspace. Store every result in reduced row-echelon form with zero rows deleted; this is a canonical representation, so duplicate fields can be recognized by exact equality. Starting with K , process the principal fields one at a time and retain both every current field and its intersection with the new one. By Proposition 5.1, the final collection is the complete subfield lattice. Explicitly forming the normal-closure Galois group is unnecessary.

5.3 Wolfram coordinate discipline

Use `FactorList` with the option `Extension -> theta` to factor over the ambient field. Coordinate conversion uses `ToNumberField` and `AlgebraicNumberPolynomial`; reduction uses `PolynomialRemainder`. The remaining computations use exact matrix routines [9, 7, 8, 10, 12]. One implementation detail is easy to get wrong. For a nonintegral primitive element, `ToNumberField` may change the generator. Coordinates relative to that new generator cannot be used as coordinates relative to the old one.

The package avoids this by choosing an integral primitive element. If the primitive integral minimal polynomial of the original input is $cx^n + a_{n-1}x^{n-1} + \dots + a_0$, take $\theta = c\alpha$. Substituting $\alpha = \theta/c$ and multiplying by c^{n-1} produces a monic integer polynomial for θ , and $\mathbb{Q}(\theta) = \mathbb{Q}(\alpha)$. For integral θ , the documented representation preserves θ [7]. The code nevertheless checks the returned generator before accepting coordinates. Rationals are handled separately, including the degree-one ambient field, where one must avoid evaluating 0^0 in a power basis.

For the two nonic examples, exact factorization over their own fields has factor degrees

$$\{1, 2, 2, 4\},$$

and the recovered subfield degrees are

$$\{1, 3, 3, 9\}. \quad (15)$$

The two cubic fields are $\mathbb{Q}(a)$ and $\mathbb{Q}(b)$. The independent implementation reproduced these results and checked that each reported basis contains one and is closed under multiplication of basis elements.

There is a useful structural interpretation. Over $\mathbb{Q}(a)$, the three conjugates $a + b_j$ have a cubic whose quadratic coefficient is $-3a$, so its trace recovers a . The three products ab_j satisfy $x^3 - a^2x + a^3$, whose coefficients also recover a , since $a = -a^3 - 1$. The full matrix algorithm generalizes this observation without requiring us to recognize the blocks or the desired cubics first.

6 A complete additive algorithm

6.1 All numbers in one finite ambient field

Fix a finite field K containing α , and let $\mathcal{F}_d(K)$ be the set of its subfields of degree at most d over $\mathbb{Q}$. Define the rational vector space

$$V_d(K) = \sum_{E \in \mathcal{F}_d(K)} E \subseteq K.$$

The plus sign here is a vector-space sum; it is not a field compositum.

Theorem 6.1 (Additive membership criterion). *There is a decomposition $\alpha = \sum_i \beta_i$, with $\beta_i \in K$ and $D(\beta_i) \leq d$, if and only if $\alpha \in V_d(K)$.*

Proof. For a proposed decomposition, each $\mathbb{Q}(\beta_i)$ is one of the fields in $\mathcal{F}_d(K)$, proving necessity. Conversely, expressing α in the vector-space sum gives one element of each of finitely many subfields. Each element's degree is at most its field's degree. $\square$

Concatenate row bases for all fields of degree at most d , transpose, and solve the rational linear system for the coordinate vector of α . If it is inconsistent, there is no such sum in K , regardless of how many terms are allowed. If it is consistent, group the solution coefficients according to their subfield to obtain the terms. Under-determined systems are harmless: any exact particular solution is sufficient [12].

Now increase d through the distinct subfield degrees. Membership changes only at those thresholds. The first successful threshold is the true minimum for arbitrary finite sums in K . This is not a greedy recursive splitting algorithm, and it does not assume that an optimal sum has two terms.

6.2 Presentation and number of terms

The linear system may return shifted or scaled generators rather than the prettiest roots. A safe additive cleanup is trace centering. If β has primitive minimal polynomial $c_d x^d + c_{d-1} x^{d-1} + \dots$, then its mean conjugate is the rational number $-c_{d-1}/(dc_d)$. Subtract that rational from the term, and collect all subtracted means into one rational offset. This preserves the sum and cannot increase a term's degree. In the supplied example it removes the rational shifts and yields the two trace-zero cubic roots.

Minimizing the number of nonzero terms is a different optimization. After fixing the best degree d , one could enumerate subsets of $\mathcal{F}_d(K)$ to find the smallest collection whose vector-space sum contains α . That finite combinatorial search is not performed by the package.

For example,

$$\eta = \sqrt{2} + \sqrt{3} + \sqrt{6}, \quad f_\eta(x) = x^4 - 22x^2 - 48x - 23, \quad (16)$$

has optimal degree two, with three natural quadratic terms. Inside the biquadratic field $\mathbb{Q}(\sqrt{2}, \sqrt{3})$, its three nonrational character components cannot be supplied by only two quadratic subfields. Nor can two external quadratic fields help: if two quadratic terms produced the primitive degree-four η , their compositum would have degree four and equal $\mathbb{Q}(\eta)$, putting them back in that same field. This is an explicit reason not to substitute a binary additive search for Theorem 6.1.

6.3 Descent to a normal closure

The internal field $\mathbb{Q}(\alpha)$ is not always enough. For sums there is nevertheless a finite ambient field that is always enough.

Theorem 6.2 (Normal-closure descent for sums). *Let $L/\mathbb{Q}$ be finite Galois and contain α . If $\alpha = \sum_{i=1}^r \beta_i$ with algebraic β_i , then there are $\gamma_i \in L$ satisfying*

$$\alpha = \sum_{i=1}^r \gamma_i, \quad \gamma_i \in L \cap \mathbb{Q}(\beta_i), \quad D(\gamma_i) \leq D(\beta_i).$$

In particular, global additive minimization can be performed in the normal closure of $\mathbb{Q}(\alpha)$.

Proof. Choose a finite Galois extension $M/\mathbb{Q}$ containing L and all β_i , and write $G = \text{Gal}(M/\mathbb{Q})$, $H = \text{Gal}(M/L)$. Since $L/\mathbb{Q}$ is Galois, H is normal in G . Define the averaging map

$$\mathcal{P}(v) = \frac{1}{|H|} \sum_{h \in H} h(v) = \frac{\text{Tr}_{M/L}(v)}{[M : L]}.$$

Its values lie in L , it fixes L pointwise, and it is $\mathbb{Q}$ -linear. Normality of H gives, for $g \in G$,

$$g\mathcal{P}(v) = \frac{1}{|H|} \sum_{h \in H} (ghg^{-1})g(v) = \mathcal{P}(gv).$$

Thus $\mathcal{P}$ commutes with every $g \in G$.

Set $J_i = \text{Gal}(M/\mathbb{Q}(\beta_i))$. Every member of J_i fixes β_i , so equivariance implies that it fixes $\gamma_i = \mathcal{P}(\beta_i)$. The Galois correspondence yields $\gamma_i \in \mathbb{Q}(\beta_i)$, while the construction already gives $\gamma_i \in L$. Hence its degree is no larger. Finally, $\sum_i \gamma_i = \mathcal{P}(\alpha) = \alpha$. Zero terms may be omitted. $\square$

The normality assumption is essential in the equivariance step. Averaging over $\text{Gal}(M/\mathbb{Q}(\alpha))$ without normality does not give the same degree-preservation conclusion.

6.4 Constructing and optimizing in that field

An exact global procedure is now finite: construct the normal closure, compute its finitely many subfields, and apply the linear membership test. In Wolfram Language, a common primitive element for all roots of the minimal polynomial can be obtained using

```
roots = Table[Root[Function[Evaluate[f /. x -> Slot[1]]], j],
  {j, Exponent[f, x]};
common = ToNumberField[roots, All];
theta = common[[1, 1]];
```

The `All` argument matters: the documentation guarantees the smallest common extension with `All`, while the default can use a larger one [7]. The number field generated by all these roots is the required splitting field.

The package first tries the smaller input field. It avoids the normal closure when the candidate meets the universal lower bound, or when the input field is already Galois.

Otherwise the global routine constructs the closure and runs the complete additive test there. Subject to unlimited computational resources, this is an algorithm for the fully unrestricted additive question, not merely a bounded coefficient search. The normal closure can have degree as large as $n!$, because its Galois group embeds into S_n . Completeness should therefore not be confused with uniform practical efficiency.

7 Products of two factors: a linear, not nonlinear, test

Fix nonzero $\alpha \in K$ and two subfields $E, F \subseteq K$. The equation $\alpha = uv$ initially appears bilinear in the unknown coordinates of u and v . Inversion in a field makes it linear.

Theorem 7.1 (Binary multiplicative intersection). *For $\alpha \neq 0$,*

$$\alpha \in E^\times F^\times \iff (\alpha E) \cap F \neq \{0\}.$$

If $w \in E \setminus \{0\}$ and $\alpha w \in F$, then $\alpha = w^{-1}(\alpha w)$ is the required factorization.

Proof. If $\alpha = uv$, choose $w = u^{-1} \in E$; then $\alpha w = v \in F$ and is nonzero. The converse is the displayed factorization, using closure of E under inversion. $\square$

Let $e_1, \dots, e_r, f_1, \dots, f_s$ be rational bases. Solve

$$\sum_{i=1}^r c_i \alpha e_i - \sum_{j=1}^s d_j f_j = 0, \quad c_i, d_j \in \mathbb{Q}. \quad (17)$$

Any nonzero nullspace vector gives a nonzero $w = \sum_i c_i e_i$. Indeed, if $w = 0$, independence of the e_i forces all $c_i = 0$, and independence of the f_j then forces all $d_j = 0$, a contradiction. Thus no nonlinear rational-point solver or numerical optimization is needed for this binary problem.

Using a complete subfield lattice, enumerate all unordered pairs E, F in increasing order of $\max([E : \mathbb{Q}], [F : \mathbb{Q}])$. This finds the exact minimum for *two factors in K* . Every candidate is finally converted with **RootReduce**, and its actual degree and exact product are checked. A rational sign adjustment makes odd-degree factors have positive constant coefficient. The implementation also normalizes a monic odd-degree constant coefficient to one when, and only when, its required root is rational. Compensating in the last factor preserves the product. This rationality check is essential: an arbitrary algebraic rescaling is not permitted.

For the degree-nine product, the two cubic subfields make (17) singular, yielding the recovery factors of Section 1. The degree-three answer meets Theorem 3.1, so this restricted search happens to certify a fully global optimum.

A binary optimum alone is not sufficient for arbitrary flat products. For example, every product of two quadratic numbers has degree at most four, while Section 8 exhibits a degree-eight number that is a product of three quadratic numbers. Also, a binary optimum in $\mathbb{Q}(\alpha)$ can exceed the global binary optimum; see Section 9.

8 Several factors and the exact tensor test

8.1 Independent field families

Suppose $E_1, \dots, E_r \subseteq K$ have degrees $d_1, \dots, d_r$, and the multiplication map

$$E_1 \otimes_{\mathbb{Q}} \cdots \otimes_{\mathbb{Q}} E_r \longrightarrow K, \quad u_1 \otimes \cdots \otimes u_r \longmapsto u_1 \cdots u_r \quad (18)$$

is an isomorphism. Equivalently, their product basis spans K and $\prod_i d_i = [K : \mathbb{Q}]$. This is the precise independence condition needed here; pairwise trivial intersections alone are not used as a substitute.

Choose bases $e_{i,1}, \dots, e_{i,d_i}$. Express α uniquely as

$$\alpha = \sum_{j_1, \dots, j_r} c_{j_1 \dots j_r} e_{1,j_1} \cdots e_{r,j_r}.$$

Then α is a product $u_1 \cdots u_r$, $u_i \in E_i$, if and only if the rational coefficient tensor c is a pure tensor, also called a rank-one tensor. This is an equivalence because (18) is injective as well as surjective.

Proposition 8.1 (Pivot certificate for a pure tensor). *Let $c \neq 0$, and choose a multi-index $p = (p_1, \dots, p_r)$ with $c_p \neq 0$. Put*

$$v_{i,j} = \frac{c_{p_1, \dots, p_{i-1}, j, p_{i+1}, \dots, p_r}}{c_p}.$$

The tensor is rank one if and only if every entry satisfies

$$c_{j_1 \dots j_r} = c_p \prod_{i=1}^r v_{i,j_i}. \quad (19)$$

When it does, take $u_i = \sum_j v_{i,j} e_{i,j}$, absorbing c_p into the first factor.

Proof. If c is a product of vectors, a nonzero pivot means each selected vector coordinate is nonzero. Dividing the appropriate slice by the pivot normalizes that vector's selected coordinate to one, which gives the formula. Conversely, (19) explicitly writes c as a product of vectors and hence supplies the factors. $\square$

The test is exact and inexpensive after the basis change. One should not replace it by a floating-point tensor-rank estimate. For more than two factors, general tensor rank is a different problem; here only rank one is being tested, with the complete entrywise certificate above.

8.2 A three-factor example that defeats binary minimization

Consider

$$\xi = (1 + \sqrt{2})(1 + \sqrt{3})(1 + \sqrt{5}). \quad (20)$$

Its minimal polynomial, verified exactly, is

$$\begin{aligned} f_\xi(x) = & x^8 - 8x^7 - 256x^6 - 768x^5 + 3008x^4 \\ & + 6144x^3 - 16384x^2 + 4096x + 4096. \end{aligned} \quad (21)$$

The multiquadratic field $\mathbb{Q}(\sqrt{2}, \sqrt{3}, \sqrt{5})$ has degree eight. Its standard eight radical monomials are linearly independent, and (20) has a nonzero coefficient on every one. No nonidentity sign-change automorphism fixes it, so it generates the full field. This also proves degree eight independently of the polynomial computation.

The three natural quadratic fields satisfy (18); the coefficient tensor is the product of the three vectors $(1, 1)$. Consequently $D_\times(\xi) = 2$. A product of two quadratic factors is impossible, since their compositum has degree at most four. The full field has sixteen subfields: seven quadratic, seven quartic, $\mathbb{Q}$, and itself, as follows from the subspaces of its elementary abelian Galois group. The independent implementation recovered all sixteen.

8.3 Exactly what the package searches

At each degree threshold, `ProductDecomposition` first runs the binary intersection tests. It then considers families of at least three distinct proper nonrational subfields, tests whether their degrees multiply to $[K : \mathbb{Q}]$, checks that their product basis has full rank, and applies (19). A family needs at most $\lfloor \log_2 [K : \mathbb{Q}] \rfloor$ members. Repeating the same subfield is unnecessary because factors in one field can be multiplied together.

With complete family enumeration, the result is a true minimum over the union of the binary and independent-family classes in $K = \mathbb{Q}(\alpha)$. It does not cover arbitrary overlapping families of three or more fields, nor factors outside K . The option "`MaxTensorFamilies`" limits candidate families; if it truncates a relevant search, the result explicitly loses the corresponding restricted-optimality claim. Meeting the universal lower bound still certifies a global optimum independently of that truncation.

9 Why restricting to the input field can miss the optimum

Consider the quartic

$$h(t) = t^4 - t - 1$$

and let r_1, r_2 be its two real roots. Define $s = r_1 + r_2$ and $u = r_1 r_2$. Then

$$f_s(x) = x^6 + 4x^2 - 1, \quad f_u(x) = x^6 + x^4 - x^3 - x^2 - 1. \quad (22)$$

These sextics provide counterexamples for *both* operations:

$$D_+^{\mathbb{Q}(s)}(s) = 6 > 4 = D_+(s), \quad D_\times^{\mathbb{Q}(u)}(u) = 6 > 4 = D_\times(u). \quad (23)$$

The superscript denotes the restriction that every term or factor belong to the indicated field, with arbitrarily many entries allowed.

9.1 Deriving the sextics

Let the remaining roots be r_3, r_4 , and set $v = r_3 r_4$. Because the total root sum is zero, factorization of h gives

$$h(t) = (t^2 - st + u)(t^2 + st + v).$$

Comparison of coefficients yields

$$u + v = s^2, \quad s(u - v) = -1, \quad uv = -1.$$

Squaring the middle equation and using the other two gives $s^2(s^4 + 4) = 1$, proving the first polynomial in (22). Substitute $v = -1/u$ and $s^2 = u - 1/u$ into the squared equation to get $(u - 1/u)(u + 1/u)^2 = 1$; multiplication by u^3 gives the second.

9.2 Galois group and field lattice

Modulo two, h is irreducible: it has no linear factor and is not the square of the only irreducible quadratic $t^2 + t + 1$. Modulo seven, its factorization has degrees one and three. These squarefree reductions imply a four-cycle and a three-cycle in the Galois group, respectively, by the standard factorization-cycle correspondence [3, Chapter 4]. Thus its group has order divisible by twelve. It cannot be the index-two subgroup A_4 , which has no four-cycle, so the group is S_4 . Its discriminant is -283 , consistently with these reductions.

The six unordered pairs of roots have distinct sums and distinct products. Two overlapping pairs with the same sum or product would force equal roots (the roots are nonzero). Equality for complementary pairs would give $s = 0$ in the sum case or $u = v$ in the product case; either contradicts $s(u - v) = -1$. Hence the stabilizer of each pair sum or pair product is exactly

$$H = \langle (12), (34) \rangle \simeq S_2 \times S_2,$$

which has order four and index six. This proves that both sextics in (22) are minimal polynomials.

Subfields of either degree-six field correspond to overgroups of H in S_4 . Apart from H and S_4 , the only such overgroup is its order-eight normalizer D_8 . To see this, a proper intermediate overgroup can have order eight or twelve. An order-twelve group would be A_4 and cannot contain the transpositions in H . In an order-eight overgroup, H has index two and is normal, so that overgroup is contained in its order-eight normalizer and must equal it.

Consequently each sextic field has precisely the subfield degrees 1, 3, 6, with a unique proper nontrivial cubic subfield. Every element of degree below six in it belongs to that cubic. Any finite sum or product of such elements remains in the cubic and cannot equal a primitive sextic element. This proves the internal optima in (23).

On the other hand, $s = r_1 + r_2$ and $u = r_1 r_2$ already use only quartic numbers. The core of H in S_4 is trivial, so the normal closure of either sextic field has group S_4 . Proposition 3.2 rules out maximum degree at most three. The global optima are therefore exactly four.

9.3 Practical consequences

The subfield lattice of $\mathbb{Q}(\alpha)$ is a powerful first search space, not an automatic global completeness certificate. In the sextic sum case, `AdditiveDecomposition` correctly reports internal degree six, whereas `GlobalAdditiveDecomposition` must pass to the degree-24 normal closure to attain the global degree four. The latter, expensive test is included as an optional example rather than described as executed Wolfram output.

There is also no justified shortcut obtained by replacing the averaging map in Theorem 6.2 with a norm. For a product $\alpha = \prod_i \beta_i$, the norm construction yields

$$\alpha^{[M:L]} = \prod_i \text{Nm}_{M/L}(\beta_i).$$

This is a factorization of a *power* of α , not of α . Extracting roots can increase degrees, leave L , and introduce root-of-unity ambiguities. The additive descent proof therefore does not establish the corresponding global product algorithm. No such unrestricted product-completeness claim is made for this implementation.

10 A complete finite-box search for either operation

A complementary approach searches defining polynomials directly. Unlike inverse coefficient matching over a chosen template, the following procedure has a simple and explicit bounded completeness guarantee.

Definition 10.1. For an algebraic number β , let $H(\beta)$ be the largest absolute coefficient of its primitive integer minimal polynomial with positive leading coefficient. This is a coefficient height, not the logarithmic Weil height. Fix positive integers d, H, k .

Enumerate every primitive irreducible integer polynomial

$$a_j x^j + \cdots + a_0, \quad 1 \leq j \leq d, \quad 1 \leq a_j \leq H, \quad |a_i| \leq H.$$

Include every exact root of every such polynomial in the catalog. Nonmonic polynomials must be included: excluding them would discard nonintegral algebraic numbers and many rational numbers.

For each candidate maximum degree $e = 1, \dots, d$, use the catalog entries of degree at most e . For each length $\ell \leq k$, enumerate multisets of the first $\ell - 1$ terms or factors. Recover the last entry exactly as

$$\beta_\ell = \alpha - \sum_{i < \ell} \beta_i \quad \text{or} \quad \beta_\ell = \frac{\alpha}{\prod_{i < \ell} \beta_i}.$$

Zero factors are excluded before division. Test the residual's actual minimal-polynomial degree and height, then verify the complete identity. The singleton case uses an empty partial list and simply tests α .

Proposition 10.2 (Finite-box completeness). *If a representation has length at most k , every entry has degree at most d , and every entry has height at most H , the search above finds a representation. Searching degree thresholds increasingly finds the smallest maximum degree among all representations in that box.*

Proof. The normalized primitive minimal polynomial of every entry occurs in the finite enumeration, and the correct conjugate occurs among its roots. Choose all but one of the entries of a proposed representation; their multiset is enumerated, and the residual is exactly the remaining entry. Conversely, every accepted residual is checked against all three bounds, and the identity is checked exactly. Thus the search is exhaustive in the specified class and introduces no false positive. $\square$

For the supplied examples one may use

```
BoundedDecompositionSearch[ap, "Product", 3, 1, 2]
BoundedDecompositionSearch[as, "Sum", 3, 1, 2]
```

because both cubics have coefficient height one. A degree-three answer then meets the global lower bound and proves global optimality. This is a second, independent algorithmic route, although exact residual minimal-polynomial computations can be substantially more expensive than their modest-looking catalog sizes suggest.

Increasing the boxes eventually finds every existing finite algebraic representation. This is a positive semidecision procedure for a fixed degree threshold: it halts when a representation exists, but an endless search does not prove nonexistence. Nor does the first result from a growing box automatically have globally minimum degree. It may precede a lower-degree representation of larger height or length. A separate lower bound is needed to turn such a result into a global optimum.

The routine reports `Missing["NoDecompositionWithinBounds", ...]` only after completing the requested finite box. Resource limits report `Failure`, not a mathematical nonexistence result. For M catalog entries, the number of multisets of $\ell - 1$ entries is $\binom{M+\ell-2}{\ell-1}$; even small degree bounds can therefore produce large searches.

11 Implementation contracts, usage, and resource limits

11.1 Public functions

The package in Appendix A is also supplied as the separate file `RootDecomposition.wl`. Its main entry points are summarized below.

Function	Contract
<code>AlgebraicDegree[a]</code>	Exact absolute degree; rejects inexact or nonalgebraic input.
<code>DegreeLowerBound[a]</code>	Largest prime divisor of the input degree, with rational case one.
<code>FieldDecompositionData[a]</code>	Power-basis coordinates and every subfield of $\mathbb{Q}(a)$, unless a resource guard stops the calculation.
<code>AdditiveDecomposition[a]</code>	Minimum maximum degree for arbitrary finite sums in $\mathbb{Q}(a)$.
<code>AdditiveDecomposition[a,data]</code>	Same additive optimization in an explicitly supplied ambient field containing a .
<code>GlobalAdditiveDecomposition[a]</code>	Global additive optimization by normal-closure descent when the smaller field is insufficient.
<code>ProductDecomposition[a]</code>	Complete binary search in $\mathbb{Q}(a)$, augmented by the independent-family tensor search; global only when separately certified.
<code>BoundedDecompositionSearch</code>	Complete degree–height–length box search for the selected operation.
<code>VerifyDecomposition</code>	Exact identity test for a supplied list and operation.

Function	Contract
<code>DecompositionExpression</code>	Displays an inactive sum or product while preserving the separate entries.

The resultant helps `RootSumPolynomial` and `RootProductPolynomial` implement (10)–(11). Their arguments are univariate rational polynomials in a clear symbol. For interpreting their outputs as monic products over roots, first normalize the input polynomials to monic form.

A successful result contains

```
<|"Operation" -> "Product", "Terms" -> {...},
  "Degrees" -> {...}, "MaximumDegree" -> ...,
  "LowerBound" -> ..., "IdentityVerified" -> True,
  "GlobalOptimalityCertified" -> ...,
  "OptimalityScope" -> ..., "Method" -> ...|>
```

A false `"GlobalOptimalityCertified"` means “not certified by this run,” not “proved nonoptimal.” The recorded scope can still establish an exact internal or bounded minimum. Conversely, when the candidate’s maximum degree equals `"LowerBound"`, the global flag is true even if the discovery method had a restricted search space. That implication comes from Theorem 3.1, not from search completeness.

11.2 Reusing an ambient field

Subfield data can be reused for additive queries in the same field:

```
data = FieldDecompositionData[theta];
r1 = AdditiveDecomposition[alpha1, data];
r2 = AdditiveDecomposition[alpha2, data];
```

Each target is checked for membership when coordinates are requested. A Galois ambient field automatically makes an internally optimal sum globally optimal by Theorem 6.2. Otherwise only an independent lower bound or a subsequent normal-closure computation supplies that claim.

For the two nonics, inspection is as simple as

```
dp = FieldDecompositionData[ap];
ds = FieldDecompositionData[as];
{Length /@ dp["Subfields"], Length /@ ds["Subfields"]}
(* Mathematical target: {{1,3,3,9}, {1,3,3,9}}. *)
```

Data matrices use a fixed power basis. Replacing the primitive element without converting the matrices changes their meaning; it is not a harmless presentation change.

11.3 Resource guards and asymptotic bottlenecks

Default field limits are `"MaxFieldDegree" -> 64` and `"MaxSubfields" -> 4096`. The tensor search defaults to `"MaxTensorFamilies" -> 10000`. The bounded search limits polynomial enumeration, catalog size, and tuple enumeration separately. Appropriate limits can be raised or set to `Infinity`, but that does not make a large problem inexpensive.

The main field bottlenecks are exact factorization over a number field, rational coefficient growth in the nullspaces, and the number of intermediate fields. Equation (13) has n columns and at most $n \deg g$

rows; all entries are rational, but their bit sizes can grow. Enumerating many canonical intersections can dominate the later linear membership tests. Partition representations from [2] are a natural optimization for large subfield lattices.

The degree guard for a normal closure is checked after a primitive element has been constructed. Thus it is not a time or memory bound on `ToNumberField[roots, All]` itself. Users facing large splitting fields should apply an outer time or memory policy as well. An aborted computation is an unknown result, never an impossibility certificate.

The reference code favors explicit verification over speed. Potential improvements include caching coordinate conversions, caching multiplication matrices, using exact sparse matrices, storing subfields by partitions, pruning tensor families by field containments, and choosing a smaller primitive element to reduce coefficient heights. These are engineering improvements to the algorithms, not missing logical hypotheses in their stated mathematical guarantees.

11.4 Output quality versus degree optimality

Even a globally degree-optimal decomposition need not have small coefficients or resemble the particular cubics written in the question. The additive trace normalization and checked rational product normalization make the examples more readable, but do not solve a secondary height minimization problem. Additional criteria could be optimized only after fixing the optimal maximum degree. Examples include the number of terms, coefficient height, number of real factors, or expression size.

We allow complex algebraic entries even when the input is real. Restricting all entries to be real defines a different problem and is not imposed by the global algorithms. The two main examples and the displayed auxiliary examples already have real optimal entries, so this distinction does not change their conclusions.

12 Executed verification and reproducibility

12.1 What was actually run

The accompanying `verify_exact.py` was executed using SymPy 1.14.0. It implements the principal-field nullspaces and intersections independently, using exact algebraic-field arithmetic and rational matrices. It is not a wrapper around the Wolfram package. The run completed with all assertions passing; its recorded elapsed time was 7.82 seconds in the preparation environment. This is a SymPy verification time, not a Mathematica benchmark.

The machine-readable report `verification_results.json` records:

Check	Executed result
Both nonic resultant identities	Exact polynomial equality
Nonic irreducibility and real branches	Degree nine; one real root each
Recovery certificates	All six polynomial remainders zero
Two nonic subfield lattices	Degrees 1, 3, 3, 9 for each
Binary product and additive searches	Maximum degree three
Three-term quadratic sum	Degree bound two; three terms
Three-factor quadratic product	Degree bound two; three factors
Degree-eight field lattice	Sixteen subfields
Both external-factor sextic lattices	Degrees 1, 3, 6

Every computed subfield basis is additionally checked for containing one and for closure under multiplication of its basis elements. This is a useful independent check that mere linear subspaces have not been confused with subfields.

12.2 What was not run

The available Wolfram connector was actually invoked, but returned HTTP 404 before any kernel evaluation. Therefore the package, its local regression suite, and the optional degree-24 normal-closure example were not executed in Mathematica. The source was statically reviewed, including balanced brackets, nested comments, and strings. That check does not validate all Wolfram evaluation semantics.

The Python verification does not test the Wolfram normal-closure construction or reproduce a Mathematica transcript. The proof of Theorem 6.2 establishes the mathematical global additive algorithm; runtime behavior of its Wolfram realization remains for the supplied local suite and additional tests to assess.

12.3 Reproduction commands

With SymPy installed, run

```
python verify_exact.py
```

This regenerates the JSON report, including a new measured elapsed time. In Mathematica, execute

```
TestReport["/your/path/tests.wlt"]
```

The test file loads the package relative to itself. It includes exact identities, resultants, degree and branch tests, all recovery congruences, subfield recovery, rational and zero cases, rejection of inexact input, the three-term and three-factor examples, and the internal/global distinction. The more expensive full normal-closure example is available separately in `examples.wl` as an optional commented section.

The PDF is reproducible with two `pdflatex` passes; the script `build.sh` supplies the commands.

Keep the source file `RootDecomposition.wl` beside the $\text{\LaTeX}$ source, because the full implementation below is included directly from that file. This prevents the printed source and executable source from drifting apart.

13 Conclusions

Both supplied degree-nine numbers have globally optimal decompositions into the requested cubic roots. Exact polynomial congruences certify their identities, and the largest-prime-factor obstruction forces the optimum three even for arbitrarily long sums or products outside the input field.

Subfield recovery provides a template-free route to those answers. Rational linear algebra handles all finite sums in a fixed field; normal-closure descent makes additive minimization globally complete. Binary products also have an exact linear test, while independent multi-factor families admit a rank-one tensor certificate. Finite-box enumeration gives another exact, but bounded, discovery procedure.

The product guarantees must remain qualified. Three quadratic factors can outperform every quadratic pair, and the sextic counterexamples require leaving the input field to attain their optimum. The returned certificates therefore state the scope of each optimality claim.

A Full Wolfram Language reference implementation

This is the exact companion source file. Execution status and mathematical scope are documented in Section 12; the listing is not presented as runtime-tested Wolfram output. The independent verification and the local `tests.wlt` suite are separate files in the archive.

```

1  (* RootDecomposition.wl
2     Exact, certificate-oriented reference implementation.
3     See the article and README for the scope of each optimality claim.
4     Independently checked mathematically with SymPy; not executed in a
5     Wolfram kernel in this session (the connector returned HTTP 404).
6  *)
7  BeginPackage["RootDecomposition`"];
8
9  AlgebraicDegree::usage = "AlgebraicDegree[a] gives the absolute degree over Q of an exact algebraic number.";
10 DegreeLowerBound::usage = "DegreeLowerBound[a] gives the largest prime divisor of AlgebraicDegree[a], or 1 for
    rationals.";
11 RootSumPolynomial::usage = "RootSumPolynomial[p,q,x] gives the resultant polynomial for all sums of roots of p
    and q.";
12 RootProductPolynomial::usage = "RootProductPolynomial[p,q,x] gives the resultant polynomial for all products
    of roots of p and q.";
13 FieldDecompositionData::usage = "FieldDecompositionData[a] computes rational coordinate bases for every
    subfield of Q(a).";
14 AdditiveDecomposition::usage = "AdditiveDecomposition[a] minimizes the largest term degree with all terms in Q
    (a). AdditiveDecomposition[a,data] uses a supplied ambient field.";
15 GlobalAdditiveDecomposition::usage = "GlobalAdditiveDecomposition[a] minimizes the largest term degree without
    an ambient-field restriction, using a normal closure when necessary.";
16 ProductDecomposition::usage = "ProductDecomposition[a] searches all binary factorizations in Q(a), and
    independent subfield tensor families. Check GlobalOptimalityCertified in its result.";
17 BoundedDecompositionSearch::usage = "BoundedDecompositionSearch[a,mode,d,h,k] searches primitive polynomial
    height <= h, degree <= d, and at most k terms or factors. mode is Sum or Product, written as a string.";
18 VerifyDecomposition::usage = "VerifyDecomposition[a,terms,mode] certifies an exact sum or product identity.";
19 DecompositionExpression::usage = "DecompositionExpression[result] displays the result as an inactive sum or
    product, preserving its decomposition.";
20
21 Begin["`Private`"];
22 $failureTag = Unique["RootDecompositionFailure"];
23 fail[tag_, data_<|>] := Throw[Failure[tag, data], $failureTag];
24 rationalQ[a_] := MatchQ[a, _Integer | _Rational];
25 zeroVectorQ[v_List] := AllTrue[v, TrueQ[# == 0] &];
26
27 minimalPolynomial[a_, x_] := Module[{b, p},
28   If[!FreeQ[a, _Real], fail["InexactInput", <|"Input" -> a|>]];
29   b = Quiet[Check[RootReduce[a], $Failed]];
30   p = Quiet[Check[MinimalPolynomial[b, x], $Failed]];
31   If[p === $Failed || !PolynomialQ[p, x] ||
32     !TrueQ[Exponent[p, x] >= 1] ||
33     !VectorQ[CoefficientList[p, x], rationalQ],
34     fail["NotExactAlgebraic", <|"Input" -> a|>]];
35   p
36 ];
37
38 degree[a_] := Module[{x = Unique["x"]},
39   Exponent[minimalPolynomial[a, x], x]
40 ];
41 lowerBoundFromDegree[n_Integer] := If[n == 1, 1, Max[First /@ FactorInteger[n]]];
42 AlgebraicDegree[a_] := Catch[degree[a], $failureTag];
43 DegreeLowerBound[a_] := Catch[lowerBoundFromDegree[degree[a]], $failureTag];
44
45 monic[p_, x_] := Expand[p/Coefficient[p, x, Exponent[p, x]]];
46 rootList[p_, x_] := Module[{fun, n = Exponent[p, x]},
47   fun = Function[Evaluate[p /. x -> Slot[1]]];
48   Table[Root[fun, j], {j, n}]
49 ];
50 RootSumPolynomial[p_, q_, x_Symbol] := Module[{y = Unique["y"]},
51   Expand[Resultant[p /. x -> y, q /. x -> x - y, y]]
52 ];
53 RootProductPolynomial[p_, q_, x_Symbol] := Module[

```

```

54 {y = Unique["y"], m = Exponent[q, x], reversed},
55 reversed = Expand[Sum[Coefficient[q, x, j] x^j y^(m-j), {j, 0, m}]];
56 Expand[Resultant[p /. x -> y, reversed, y]]
57 ];
58
59 canonicalRows[rows_List, n_Integer] := If[rows === {}, {},
60   Select[RowReduce[rows], !zeroVectorQ[#] &]
61 ];
62 spaceIntersection[u_List, v_List, n_Integer] := Module[{ann},
63   ann = Join[NullSpace[u], NullSpace[v]];
64   If[ann === {}, IdentityMatrix[n], canonicalRows[NullSpace[ann], n]]
65 ];
66
67 coordinates[a_, data_Association] := Module[
68   {n = data["Degree"], theta = data["Generator"],
69    x = data["Variable"], b, an, p, v},
70   b = RootReduce[a];
71   If[rationalQ[b], Return[PadRight[{b}, n]]];
72   an = Quiet[Check[ToNumberField[b, theta], $Failed]];
73   If[Head[an] != AlgebraicNumber,
74     fail["NotInAmbientField", <|"Element" -> b|>]];
75   (* Never assume that a changed primitive element uses the same coordinates. *)
76   If[!TrueQ[RootReduce[an[[1]] - theta] == 0],
77     fail["UnexpectedPrimitiveElement", <|"Returned" -> an[[1]],
78       "Expected" -> theta|>]];
79   p = AlgebraicNumberPolynomial[an, x];
80   p = PolynomialRemainder[p, data["MinimalPolynomial"], x];
81   v = PadRight[CoefficientList[p, x], n];
82   If[Length[v] != n || !VectorQ[v, rationalQ],
83     fail["InvalidCoordinates", <|"Element" -> b|>]];
84   v
85 ];
86 fromCoordinates[v_List, data_Association] := If[data["Degree"] == 1,
87   First[v], RootReduce[v . data["Generator"]^Range[0, data["Degree"]-1]]
88 ];
89 multiplyCoordinates[u_List, v_List, data_Association] := Module[
90   {x = data["Variable"], n = data["Degree"], powers, p},
91   powers = x^Range[0, n-1];
92   p = PolynomialRemainder[Expand[(u . powers) (v . powers)],
93     data["MinimalPolynomial"], x];
94   PadRight[CoefficientList[p, x], n]
95 ];
96
97 principalBasis[g_, data_Association] := Module[
98   {x = data["Variable"], theta = data["Generator"],
99    n = data["Degree"], m, columns, rem, coeff},
100   m = Exponent[g, x];
101   columns = Table[
102     rem = PolynomialRemainder[x^j - theta^j, g, x];
103     coeff = PadRight[CoefficientList[Expand[rem], x], m];
104     Flatten[coordinates[#, data] & /@ coeff],
105     {j, 0, n-1}];
106   canonicalRows[NullSpace[Transpose[columns]], n]
107 ];
108
109 Options[FieldDecompositionData] = {
110   "MaxFieldDegree" -> 64, "MaxSubfields" -> 4096
111 };
112 FieldDecompositionData[a_, OptionsPattern[]] := Catch[Module[
113   {x = Unique["x"], p, n, theta, data, fl, factors,
114    principal, fields, next, dmax = OptionValue["MaxFieldDegree"],
115    fmax = OptionValue["MaxSubfields"]},
116   p = minimalPolynomial[a, x]; n = Exponent[p, x];
117   If[n > dmax, fail["FieldDegreeLimit", <|"Degree" -> n, "Limit" -> dmax|>]];
118   (* If c is the leading coefficient of a primitive integral minimal
119     polynomial of a, c a is an algebraic integer and generates Q(a). *)
119   theta = If[n == 1, 0, RootReduce[Coefficient[p, x, n] a]];
120   p = monic[minimalPolynomial[theta, x], x];
121   data = <|"Generator" -> theta, "Variable" -> x,
122     "MinimalPolynomial" -> p, "Degree" -> n|>;
123 ]

```

```

124 If[n == 1, Return[Join[data, <|"Factors" -> {x},
125   "PrincipalFields" -> {{{1}}}, "Subfields" -> {{{1}}}],
126   "IsGalois" -> True|>]];
127 fl = Quiet[Check[FactorList[p, Extension -> theta], $Failed]];
128 If[fl === $Failed || Head[fl] != List,
129   fail["FactorizationFailed"]];
130 fl = Rest[fl];
131 If[!AllTrue[fl, Last[#] == 1 &], fail["UnexpectedMultiplicity"]];
132 factors = monic[First[#], x] & /@ fl;
133 If[Total[Exponent[#, x] & /@ factors] != n,
134   fail["IncompleteFactorization"]];
135 If[!AllTrue[CoefficientList[Expand[(Times @@ factors) - p], x],
136   TrueQ[RootReduce[#] == 0] &], fail["FactorizationCheckFailed"]];
137 principal = DeleteDuplicates[principalBasis[#, data] & /@ factors];
138 fields = {IdentityMatrix[n]};
139 Do[
140   next = DeleteDuplicates[Join[fields,
141     spaceIntersection[#, b, n] & /@ fields]];
142   If[Length[next] > fmax,
143     fail["SubfieldLimit", <|"Limit" -> fmax|>]];
144   fields = next,
145   {b, principal}];
146 fields = SortBy[fields, Function[b, {Length[b], ToString[b, InputForm]}]];
147 If[!AllTrue[fields, Length[#] > 0 && Mod[n, Length[#]] == 0 &],
148   fail["InvalidSubfieldDimensions"]];
149 Join[data, <|"Factors" -> factors, "PrincipalFields" -> principal,
150   "Subfields" -> fields, "IsGalois" -> (Length[factors] == n)|>]
151 ], $failureTag];
152
153 VerifyDecomposition[a_, terms_List, mode_String] := Module[{value},
154   If[!MemberQ[{"Sum", "Product"}, mode], Return[False]];
155   value = If[mode == "Sum", Total[terms], Times @@ terms];
156   TrueQ[RootReduce[a - value] == 0]
157 ];
158 DecompositionExpression[result_Association] :=
159   Apply[If[result["Operation"] == "Sum", Inactive[Plus], Inactive[Times]],
160     result["Terms"]];
161
162 traceMean[a_] := Module[{x = Unique["x"], p, d},
163   p = minimalPolynomial[a, x]; d = Exponent[p, x];
164   -Coefficient[p, x, d-1]/(d Coefficient[p, x, d])
165 ];
166 centerTerms[terms_List] := Module[{means, centered, c},
167   means = traceMean /@ terms;
168   centered = MapThread[RootReduce[#1 - #2] &, {terms, means}];
169   centered = Select[centered, !TrueQ[# == 0] &];
170   c = Total[means];
171   If[c != 0, AppendTo[centered, c]];
172   If[centered === {}, {0}, centered]
173 ];
174 normalizeProductFactors[terms_List] := Module[
175   {out = terms, x = Unique["x"], p, d, ratio, scale},
176   Do[
177     p = minimalPolynomial[out[[j]], x]; d = Exponent[p, x];
178     If[d > 1 && OddQ[d] && Coefficient[p, x, 0] < 0,
179       out[[j]] = RootReduce[-out[[j]]];
180       out[[-1]] = RootReduce[-out[[-1]]];
181       p = minimalPolynomial[out[[j]], x];
182       (* Normalize an odd-degree constant term only when the required
183         scaling is rational. Never introduce an algebraic scaling. *)
184       If[d > 1 && OddQ[d],
185         ratio = Coefficient[p, x, 0]/Coefficient[p, x, d];
186         scale = RootReduce[ratio^(1/d)];
187         If[rationalQ[scale] && scale != 0,
188           out[[j]] = RootReduce[out[[j]]/scale];
189           out[[-1]] = RootReduce[scale out[[-1]]],
190         {j, Max[0, Length[out]-1]}];
191     out
192   ];
193 record[a_, terms_List, mode_, method_, scope_, global_:False] := Module[

```

```

194 {ts, ds, lo},
195 ts = RootReduce /@ terms;
196 If[ts === {}, ts = {If[mode == "Sum", 0, 1]};
197 If[!VerifyDecomposition[a, ts, mode], fail["IdentityCheckFailed"]];
198 ds = degree /@ ts; lo = lowerBoundFromDegree[degree[a]];
199 <|"Operation" -> mode, "Terms" -> ts, "Degrees" -> ds,
200   "MaximumDegree" -> Max[ds], "LowerBound" -> lo,
201   "IdentityVerified" -> True,
202   "GlobalOptimalityCertified" -> (TrueQ[global] || Max[ds] == lo),
203   "OptimalityScope" -> scope, "Method" -> method|>
204 ];
205
206 sumAtDegree[a_, data_Association, d_Integer] := Module[
207   {blocks, rows, matrix, target, c, terms = {}, pos = 1, v},
208   blocks = Select[data["Subfields"], Length[#] <= d &];
209   rows = Join @@ blocks; matrix = Transpose[rows];
210   target = coordinates[a, data];
211   If[MatrixRank[Join[matrix, Transpose[{target}], 2]] != MatrixRank[matrix],
212     Return[Missing["NoSumAtThisDegree"]]];
213   c = LinearSolve[matrix, target];
214   Do[
215     v = Take[c, {pos, pos+Length[b]-1}] . b;
216     If[!zeroVectorQ[v], AppendTo[terms, fromCoordinates[v, data]]];
217     pos += Length[b],
218     {b, blocks}];
219   centerTerms[terms]
220 ];
221 minimumSum[a_, data_Association] := Module[{terms},
222   Do[
223     terms = sumAtDegree[a, data, d];
224     If[ListQ[terms], Return[record[a, terms, "Sum", "SubfieldLinearAlgebra",
225       "Minimum in the supplied ambient field", data["IsGalois"]]]],
226     {d, Sort[DeleteDuplicates[Length /@ data["Subfields"]]]];
227   fail["TargetNotInAmbientField"]
228 ];
229 Options[AdditiveDecomposition] = Options[FieldDecompositionData];
230 AdditiveDecomposition[a_, data_Association] := Catch[minimumSum[a, data], $failureTag];
231 AdditiveDecomposition[a_, OptionsPattern[]] := Catch[Module[{data},
232   data = FieldDecompositionData[a,
233     "MaxFieldDegree" -> OptionValue["MaxFieldDegree"],
234     "MaxSubfields" -> OptionValue["MaxSubfields"]];
235   If[FailureQ[data], Return[data]];
236   minimumSum[a, data]
237 ], $failureTag];
238
239 Options[GlobalAdditiveDecomposition] = Options[FieldDecompositionData];
240 GlobalAdditiveDecomposition[a_, OptionsPattern[]] := Catch[Module[
241   {data, ans, roots, common, theta, normalData},
242   data = FieldDecompositionData[a,
243     "MaxFieldDegree" -> OptionValue["MaxFieldDegree"],
244     "MaxSubfields" -> OptionValue["MaxSubfields"]];
245   If[FailureQ[data], Return[data]];
246   ans = minimumSum[a, data];
247   If[TrueQ[ans["GlobalOptimalityCertified"]], Return[ans]];
248   roots = rootList[data["MinimalPolynomial"], data["Variable"]];
249   common = Quiet[Check[ToNumberField[roots, All], $Failed]];
250   If[common === $Failed || !ListQ[common] ||
251     Head[First[common]] != AlgebraicNumber,
252     fail["NormalClosureConstructionFailed", <|"InputFieldCandidate" -> ans|>]];
253   theta = common[[1, 1]];
254   normalData = FieldDecompositionData[theta,
255     "MaxFieldDegree" -> OptionValue["MaxFieldDegree"],
256     "MaxSubfields" -> OptionValue["MaxSubfields"]];
257   If[FailureQ[normalData], Return[Failure["NormalClosureLimitOrFailure",
258     <|"Cause" -> normalData, "InputFieldCandidate" -> ans|>]]];
259   If[!TrueQ[normalData["IsGalois"]],
260     fail["NormalClosureNotGalois", <|"InputFieldCandidate" -> ans|>]];
261   ans = minimumSum[a, normalData];
262   Join[ans, <|"GlobalOptimalityCertified" -> True,
263     "OptimalityScope" -> "Global, by normal-closure trace descent",

```

```

264   "AmbientFieldDegree" -> normalData["Degree"]|>]
265 ], $failureTag];
266
267 binaryProductAtDegree[a_, data_Association, d_Integer] := Module[
268   {fields, av, e, f, ae, ns, v, u, factors},
269   fields = Select[data["Subfields"], Length[#] <= d &];
270   av = coordinates[a, data];
271   Do[
272     e = fields[[i]];
273     ae = multiplyCoordinates[av, #, data] & /@ e;
274     Do[
275       f = fields[[j]];
276       ns = NullSpace[Transpose[Join[ae, -f]]];
277       If[ns != {},
278         v = Take[First[ns], Length[e]] . e;
279         u = fromCoordinates[v, data];
280         If[TrueQ[u == 0], fail["UnexpectedZeroKernelVector"]];
281         factors = RootReduce /@ {1/u, a u};
282         Return[normalizeProductFactors[factors]],
283       {j, i, Length[fields]}],
284   {i, Length[fields]}];
285   Missing["NoBinaryProductAtThisDegree"]
286 ];
287
288 tensorProductForFamily[a_, data_Association, family_List] := Module[
289   {dims = Length /@ family, n = data["Degree"], tuples,
290     one, products, matrix, c, pivotPosition, pivotIndex, pivot,
291     tensor, vectors, predicted, factors},
292   If[(Times @@ dims) != n, Return[Missing["WrongDegreeProduct"]]];
293   tuples = Tuples[Range /@ dims]; one = UnitVector[n, 1];
294   products = Table[
295     Fold[multiplyCoordinates[#1, #2, data] &, one,
296       MapThread[Part, {family, index}]],
297     {index, tuples}];
298   matrix = Transpose[products];
299   If[MatrixRank[matrix] != n, Return[Missing["NotIndependent"]]];
300   c = LinearSolve[matrix, coordinates[a, data]];
301   pivotPosition = First[Select[Range[Length[c]], c[[#]] != 0 &, 1]];
302   pivotIndex = tuples[[pivotPosition]]; pivot = c[[pivotPosition]];
303   tensor = ArrayReshape[c, dims];
304   vectors = Table[
305     Table[Extract[tensor, ReplacePart[pivotIndex, k -> j]]/pivot,
306       {j, dims[[k]]}], {k, Length[dims]}];
307   predicted = Table[pivot (Times @@ MapThread[Part, {vectors, index}]),
308     {index, tuples}];
309   If[predicted != c, Return[Missing["TensorRankNotOne"]]];
310   factors = MapThread[fromCoordinates[#1 . #2, data] &, {vectors, family}];
311   factors[[1]] = RootReduce[pivot factors[[1]]];
312   normalizeProductFactors[factors]
313 ];
314
315 Options[ProductDecomposition] = Join[Options[FieldDecompositionData], {
316   "TensorFamilies" -> True, "MaxTensorFamilies" -> 10000
317 }];
318 ProductDecomposition[a_, OptionsPattern[]] := Catch[Module[
319   {data, n, thresholds, terms, eligible, count, families, budget,
320     spent, complete = True, useTensors = OptionValue["TensorFamilies"],
321     scope, method, found = False},
322   n = degree[a];
323   If[n == 1, Return[record[a, {RootReduce[a]}, "Product", "Rational",
324     "Global", True]]];
325   data = FieldDecompositionData[a,
326     "MaxFieldDegree" -> OptionValue["MaxFieldDegree"],
327     "MaxSubfields" -> OptionValue["MaxSubfields"]];
328   If[FailureQ[data], Return[data]];
329   thresholds = Sort[DeleteDuplicates[Length /@ data["Subfields"]]];
330   Do[
331     terms = binaryProductAtDegree[a, data, d];
332     If[ListQ[terms], method = "BinarySubfieldIntersection"; found = True];
333     If[!found && TrueQ[useTensors],

```

```

334     eligible = Select[data["Subfields"], 1 < Length[#] <= d && Length[#] < n &];
335     spent = 0; budget = OptionValue["MaxTensorFamilies"];
336     Do[
337         count = Binomial[Length[eligible], k];
338         If[count > 0,
339             If[budget == Infinity,
340                 families = Subsets[eligible, {k}],
341                 families = Subsets[eligible, {k}, Max[0, budget-spent]];
342                 If[count > Max[0, budget-spent], complete = False]];
343             spent += Length[families];
344             Do[
345                 If[(Times @@ (Length /@ family)) == n,
346                     terms = tensorProductForFamily[a, data, family];
347                     If[ListQ[terms], method = "IndependentSubfieldTensor";
348                         found = True; Break[]],
349                     {family, families}]];
350             If[found, Break[]],
351             {k, 3, Floor[Log[2, n]]}]];
352     If[found,
353         scope = Which[
354             !complete, "Candidate only; tensor-family enumeration was truncated",
355             !TrueQ[useTensors], "Minimum for two factors in Q(a)",
356             True, "Minimum over binary and independent-family products in Q(a)"];
357         Return[Join[record[a, terms, "Product", method, scope],
358             <|"TensorSearchComplete" -> complete|>]],
359         {d, thresholds}];
360     fail["InternalProductSearchFailure"]
361 ], $failureTag];
362
363 (* Exhaustive search under EXPLICIT finite bounds. The last term/factor is
364    reconstructed exactly rather than independently enumerated. *)
365 Options[BoundedDecompositionSearch] = {
366     "MaxPolynomials" -> 20000, "MaxCandidates" -> 10000,
367     "MaxTuples" -> 100000
368 };
369 BoundedDecompositionSearch[a_, mode_String, maxDegree_Integer,
370     height_Integer, maxTerms_Integer, OptionsPattern[]] :=
371 Catch[Module[
372     {x = Unique["x"], catalog = {}, p, roots, polys, candidates,
373     combinations, indices, chosen, residual, terms, mp, count, used,
374     inBox, polyBudget = OptionValue["MaxPolynomials"],
375     tupleBudget = OptionValue["MaxTuples"]},
376     If[!MemberQ[{"Sum", "Product"}, mode], fail["InvalidOperation"]];
377     If[Min[maxDegree, height, maxTerms] < 1, fail["InvalidSearchBounds"]];
378     degree[a];
379     polys = Sum[height (2 height+1)^j, {j, 1, maxDegree}];
380     If[polys > polyBudget, fail["PolynomialEnumerationLimit",
381         <|"Required" -> polys, "Limit" -> polyBudget|>]];
382     Do[
383         Do[
384             Do[
385                 If[(GCD @@ Append[Abs[cs], lead]) == 1,
386                     p = Append[cs, lead] . x^Range[0, j];
387                     If[IrreduciblePolynomialQ[p],
388                         roots = rootList[p, x];
389                         If[mode == "Product", roots = Select[roots, !TrueQ[# == 0] &]];
390                         catalog = Join[catalog, ({#, j} & /@ roots)];
391                         If[Length[catalog] > OptionValue["MaxCandidates"],
392                             fail["CandidateEnumerationLimit"]],
393                         {lead, 1, height}},
394                     {cs, Tuples[Range[-height, height], j]}],
395                     {j, 1, maxDegree}];
396     inBox[b_, d_] := Module[{f = minimalPolynomial[b, x]},
397     Exponent[f, x] <= d && Max[Abs[CoefficientList[f, x]]] <= height];
398     Do[
399         candidates = First /@ Select[catalog, Last[#] <= d &];
400         used = 0;
401         Do[
402             If[k == 1,
403                 combinations = {{}},

```

```

404     count = Binomial[Length[candidates]+k-2, k-1];
405     If[used+count > tupleBudget, fail["TupleEnumerationLimit",
406         <|"RequiredAtThisDegree" -> used+count, "Limit" -> tupleBudget|>]];
407     combinations = Subsets[Range[Length[candidates]+k-2], {k-1}];
408     used += Length[combinations];
409     Do[
410         indices = If[k == 1, {}, comb - Range[0, k-2]];
411         chosen = candidates[[indices]];
412         residual = RootReduce[If[mode == "Sum",
413             a - Total[chosen], a/(Times @@ chosen)]];
414         If[inBox[residual, d],
415             terms = Append[chosen, residual];
416             Return[Join[record[a, terms, mode, "BoundedEnumeration",
417                 "Minimum within the specified degree, height, and length box"],
418                 <|"HeightBound" -> height, "LengthBound" -> maxTerms|>]],
419             {comb, combinations}],
420         {k, 1, maxTerms}],
421         {d, 1, maxDegree}];
422     Missing["NoDecompositionWithinBounds",
423         <|"DegreeBound" -> maxDegree, "HeightBound" -> height,
424         "LengthBound" -> maxTerms|>]
425 ], $failureTag];
426
427 End[];
428 EndPackage[];

```

References

- [1] Vladimir Reshetnikov, Mathematica StackExchange question 105933, concerning products and sums of lower-degree algebraic roots; answers by yarchik and Daniel Lichtblau (2019). <https://mathematica.stackexchange.com/q/105933/7288>. The question's two examples are reproduced from the user's supplied text.
- [2] J. Szutkoski and M. van Hoeij, *The Complexity of Computing all Subfields of an Algebraic Number Field*, arXiv:1606.01140v3, 20 November 2017. <https://arxiv.org/html/1606.01140v3>.
- [3] J. S. Milne, *Fields and Galois Theory*, version 5.10, September 2022. In particular Chapters 3–5 on the Galois correspondence, Galois groups, primitive elements, norms, and traces. <https://www.jmilne.org/math/CourseNotes/FT.pdf>.
- [4] Wolfram Research, *Root*. <https://reference.wolfram.com/language/ref/Root.html>.
- [5] Wolfram Research, *RootReduce*. <https://reference.wolfram.com/language/ref/RootReduce.html>.
- [6] Wolfram Research, *MinimalPolynomial*. <https://reference.wolfram.com/language/ref/MinimalPolynomial.html>.
- [7] Wolfram Research, *ToNumberField*. <https://reference.wolfram.com/language/ref/ToNumberField.html>.
- [8] Wolfram Research, *AlgebraicNumberPolynomial*. <https://reference.wolfram.com/language/ref/AlgebraicNumberPolynomial.html>.
- [9] Wolfram Research, *FactorList*. <https://reference.wolfram.com/language/ref/FactorList.html>.
- [10] Wolfram Research, *PolynomialRemainder*. <https://reference.wolfram.com/language/ref/PolynomialRemainder.html>.
- [11] Wolfram Research, *Resultant*. <https://reference.wolfram.com/language/ref/Resultant.html>.
- [12] Wolfram Research, *LinearSolve*. <https://reference.wolfram.com/language/ref/LinearSolve.html>.
- [13] Wolfram Research, *CountRoots*. <https://reference.wolfram.com/language/ref/CountRoots.html>.
- [14] Wolfram Research, *IrreduciblePolynomialQ*. <https://reference.wolfram.com/language/ref/IrreduciblePolynomialQ.html>.

Official Wolfram documentation consulted 6 September 2026.